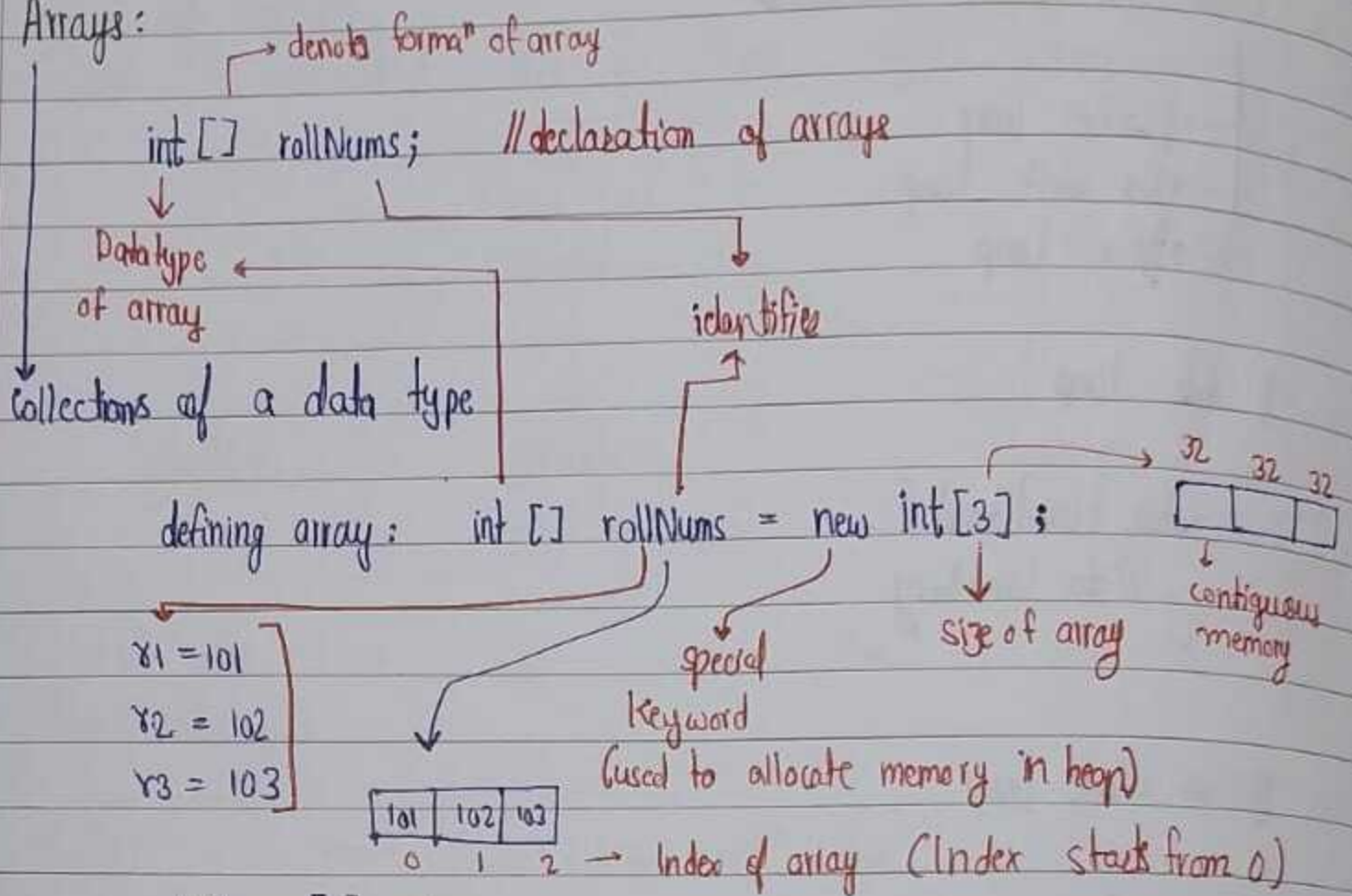


Arrays in Java

* Arrays:



`rollNums[0] = 101;`

`rollNums[1] = 102;`

`rollNums[2] = 103;`

Retrieving:

`System.out.println(rollNums[0]); // 101`

`----- (rollNums[1]); // 102`

`----- (rollNums[2]); // 103`

Using loops to assign values in array

`int x = 101;`

`for (int i = 0; i < 3; i++) {`

`rollNums[i] = x;`

`x++;`

`}`

retrieving:

`for (int i = 0; i < 3; i++) {`

`System.out.println(rollNums[i]);`

`}`

#property of array:

rollNums.length $\rightarrow$ gives size of array

use case:

```
for (i=0; i < rollNums.length; i++) {  
    System.out.println(rollNums[i]);  
}
```

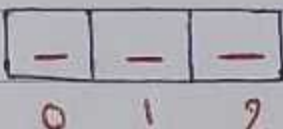
suppose:

```
System.out.println(rollNums[4]);
```

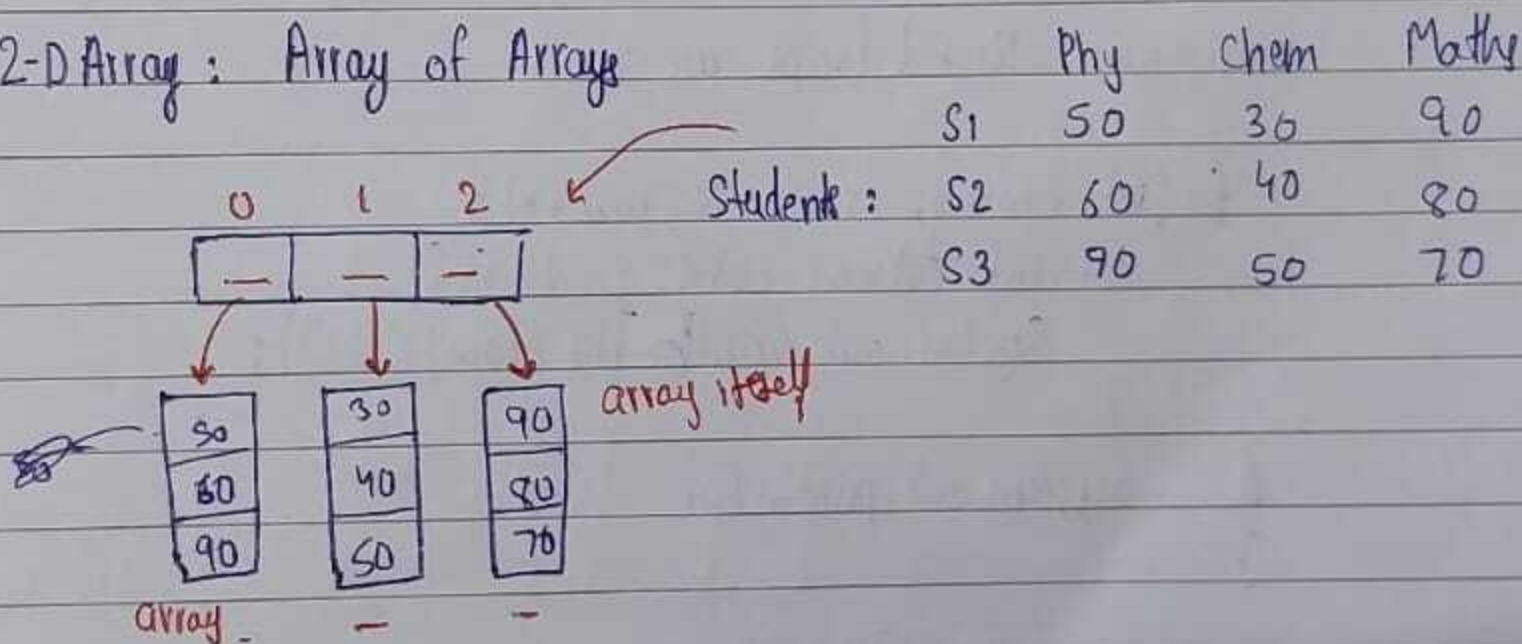

 $\rightarrow$ out of bound
 $\rightarrow$ Index out of bound exception.

* Multidimensional array

```
int [] rollNums = new int [3];
```



 $\rightarrow$ 1-D Array (Array of int in this eg)

2-D Array: Array of Arrays



Conceptual representation of 2D Array

int [][] marks = new int [3][3];

		Marks								
Visualisation:	Students:	S ₁	50	30	90	— x ₁	0 x ₁	(0,0)	(0,1)	(0,2)
			S ₂	60	40	80	— x ₂	1 x ₂	(1,0)	(1,1)
		S ₃	70	50	70	— x ₃	2 x ₃	(2,0)	(2,1)	(2,2)
		P	C	M	(3x3)					
		c ₁	c ₂	c ₃						

here we need to use nested loop to fill values with loops

marks [0][0] = 50;

marks [0][1] = 30;

marks [0][2] = 90;

marks [1][0] = 60;

marks [1][1] = 40;

marks [1][2] = 80;

⋮

retriving: Nested loops use case

```

for(int row=0; row<3; row++){
    for(int col=0; col<3; col++){
        System.out.print(marks [row] [col]);
    }
    System.out.println();
}

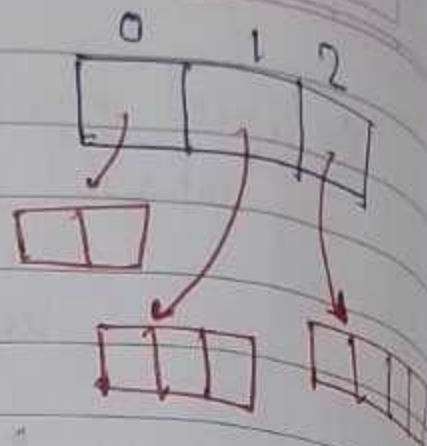
```

// 50 30 90

// 60 40 80

// 70 50 70

$\text{int}[][] \text{marks} = \text{new int}[3][\quad];$
 row column
 compulsory optional



$\text{marks}[0] = \text{new int}[2];$
 $\text{marks}[1] = \text{new int}[3];$
 $\text{marks}[2] = \text{new int}[4];$

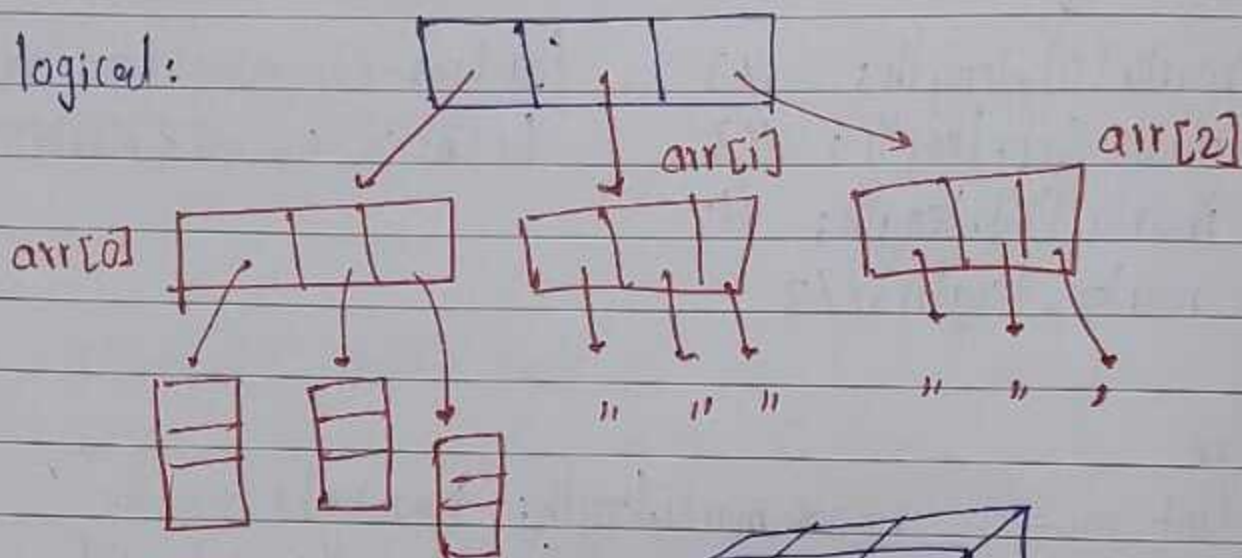
representation: →

		c_0	c_1	c_2	c_3	c_4
0 r_1	(0,0)	(0,1)				
1 r_2	(1,0)	(1,1)	(1,2)			
2 r_3	(2,0)	(2,1)	(2,2)	(2,3)		

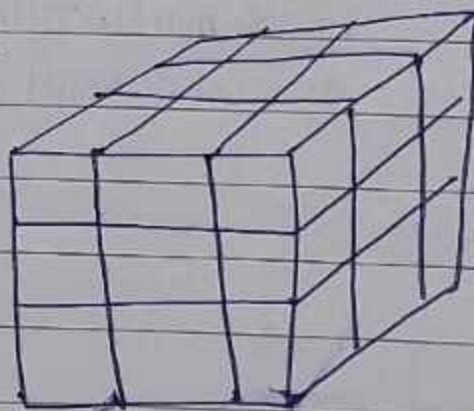
#3D Array:

$\text{int}[][][] \text{arr} = \text{new int}[3][3][3];$

logical:



Conceptual representation:



Cube

[it is like
 coordinates in
 3D space
 (x, y, z)]

4 Other way of Array Declaration

```
int [] roll-Nums = {4, 5, 6}; // 1D Array
```

```
int [][] marks = { {4, 5, 6}, {7, 8, 9}, {23, 34, 21} } // 2D Array
```

Strings in Java
non primitive
data type

non primitive → Array, String, ... and a lot more.

```
String fname = "Aditya"; (in double quote)  
String lname = "Raj";
```

```
System.out.println("Hello");  
String
```

```
System.out.println(fname + lname);
```

String will be studied more in depth in future once our foundations becomes more strong.